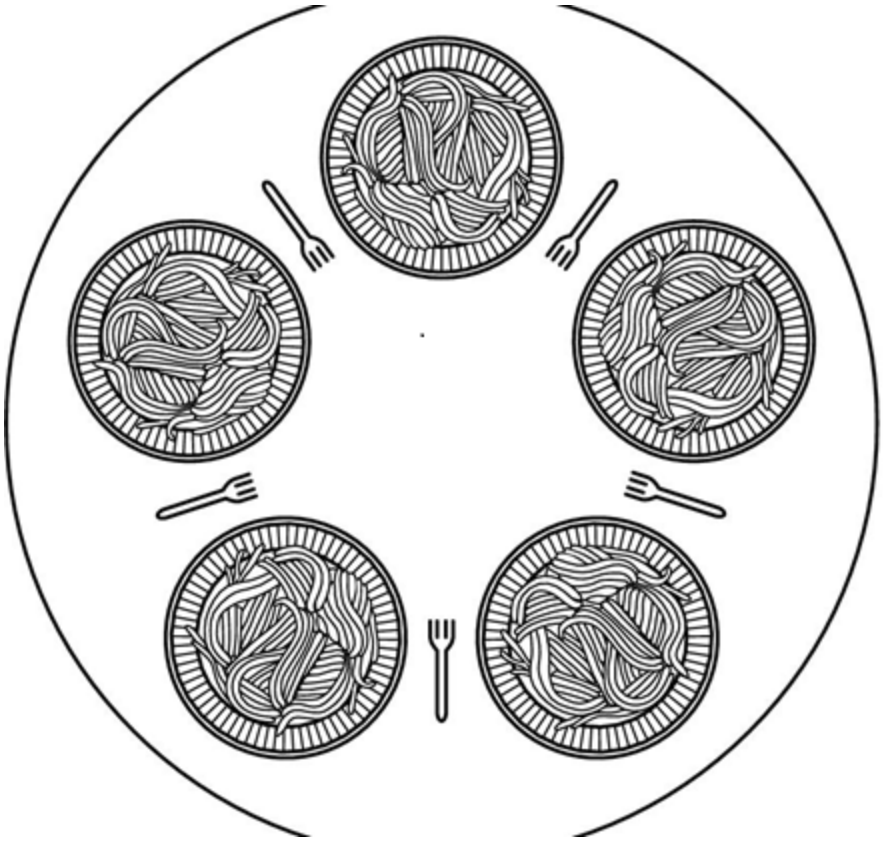
Write a c/c++ program to implement the dining philosophers problem on kylin desktop os v10

Target

1. Write a c/c++ program on kylin desktop os v10
2. To implement the dining philosophers problem
3. Gcc



Tools

Install GCC Software Colletion

```
sudo apt-get install build-essential
```

How to use GCC

- gcc and make

posix thread

```
#include <pthread.h>
pthread_create()
```

How to do

write a c program to implement the dining philosophers problem

The specific requirements are as follows

- 5个哲学家就餐思考问题
- 思考随机时间3~8s
- 就餐随机时间2~10s
- ***就餐10次后结束***

```
#define N  5
void philosopher(int i){
  while(TRUE){
      think();
      take_fork(i);
      take_fork((i+1)%N);
      eat();
      put_fork(i);
      put_fork((i+1)%N);
  }
}
```

```
#define N 5
#define LEFT (i+N-1)%N
#define RIGHT (i+1)%N
#define THINKING  0
#define  HUNGRY  1
#define  EATING    2
typedef int semaphore;
int state[N];
semaphore mutex=1;
semaphore s[N];
void philosopher(int){
  while(TRUE){
     think();
           take_forks(i);
     eat();
        put_forks(i);
  }
}


void take_forks(int i){
  down(&mutex);
  state[i]=HUNGRY;     test(i);
  up(&mutex);
  down(&s[i]);
}
void put_forks(int i){
  down(&mutex);
  state[i]=THINKING;
  test(LEFT);     test(RIGHT);
  up(&mutex);
}
void test(int i){
  if(state[i]==HUNGRY && state[LEFT]!=EATING && state[RIGHT]!=EATING){
    state[i]=EATING;
    up(&s[i]);
  }
}
```

Example of multi-threads

```
#include<stdio.h>
#include <pthread.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>

typedef struct ST_ARGS
{
    int id;
    char name[32];
}ARGS;

void *ThreadFunc(void *args)
{
    ARGS* pArgs=(ARGS*)args;
    usleep(1000+rand()%1000);
    printf ("thread id = %d\n", pArgs->id);
}

int main(void)
{
    int     err;
    pthread_t tid;
    int     i=0;
    srand((unsigned)time(NULL));
    while (1)
    {
        i++;
        if (i>9) break;
        ARGS* pmyargs=(ARGS*)malloc(sizeof(ARGS));
        pmyargs->id=i;
        err= pthread_create(&tid, NULL, ThreadFunc, pmyargs);
        if(err != 0){
            printf("can't create thread: %s\n",strerror(err));
            break;
        }else
        {
            printf("create thread(%ld)\n",tid);
        }
        usleep(2000);
```

```
    }
    return 0;
}
```

```
gcc pthread_test.c  -o pthread_test  -lpthread
./pthread_test
```